Ludoteka Campus
Colegio Francisco Arango

INSTALACIÓN Y DESPLIEGUE

Manual de Instalación

Cómo preparar el entorno, configurar los servicios y poner en marcha Ludoteka Campus en desarrollo, Docker o producción.



EDICIÓN

Versión 1.0 · octubre de 2026

EQUIPO

Darwin Joan Aveiga Mora · Arquitectura y desarrollo full stack
Mariana Sofía Azabache Moreno · Project manager, diseño UI/UX y frontend
Julián Esteban Chávez Zamora · QA y apoyo en backend

Versión del documento	1.0, octubre de 2026
Para quién	Personal de sistemas que instala Ludoteka Campus en un ambiente nuevo o lo prepara para desarrollo
Tiempo estimado	1 a 2 horas, sin contar la verificación del dominio de correo

Este manual instala el sistema desde cero: base de datos, servicios externos, servidor y primera cuenta de administración. Para entender el sistema antes de instalarlo, lea el Manual técnico.

Contenido

1. Qué se necesita
2. Base de datos en CockroachDB Cloud
3. Servicios externos
4. Instalación local para desarrollo
5. Instalación con Docker
6. Despliegue en Render
7. Primer ingreso y puesta en marcha
8. Verificación
9. Dominio propio
10. Problemas frecuentes al instalar

1. Qué se necesita

Cuentas:

Servicio	Para qué	¿Obligatorio?
GitHub, con acceso al repositorio	Código fuente y despliegue automático	Sí
CockroachDB Cloud	Base de datos (es la única soportada)	Sí
Render	Servidor	Sí, o cualquier hospedaje con Node.js y HTTPS
Resend	Correo: códigos de ingreso, claves temporales, avisos	Sí en producción: sin correo nadie puede terminar de ingresar
Cloudinary	Archivos de la biblioteca	Para subir archivos
Google AI Studio	Clave de Gemini para el tutor, el Observatorio y la búsqueda semántica	No: sin clave, cada función usa su versión sin IA

En el equipo de quien instala (solo para desarrollo o para correr scripts):

- Git.
- Node.js 22 (mínimo 20). Compruebe con `node -v`.
- Un editor de código.

2. Base de datos en CockroachDB Cloud

1. Entre a CockroachDB Cloud y cree un clúster (**Create cluster**). El plan Basic sirve para empezar.
2. Use una versión que soporte índices vectoriales. Producción usa v26.2.
3. Cree un usuario SQL con contraseña (**SQL Users**). El usuario debe ser administrador del clúster; el que crea el asistente inicial lo es. El Observatorio lo necesita para cambiar al rol de solo lectura `ludoteka_reader`.
4. En **Connect**, copie la cadena de conexión general. Tiene esta forma:

   ```
   postgresql://USUARIO:CONTRASEÑA@HOST:26257/defaultdb?sslmode=verify-full
   ```

   Esa cadena es `DATABASE_URL`. Guárdela en un gestor de secretos; **no** la pegue en documentos ni chats.

No hace falta crear tablas: la API aplica las 36 migraciones de `db/migrations` la primera vez que arranca.

Si la migración `0011_library.sql` falla por el índice vectorial, habilite la función en la consola SQL del clúster y vuelva a desplegar:

```
SET CLUSTER SETTING feature.vector_index.enabled = true;
```

3. Servicios externos

Resend (correo)

1. Cree una cuenta en Resend.
2. En **Domains**, agregue el dominio del colegio y cree en su DNS los registros que Resend indique. Espere a que el dominio quede verificado.
3. En **API Keys**, cree una clave con permiso de envío. Es `RESEND_API_KEY`.
4. Defina el remitente `MAIL_FROM`, por ejemplo `Ludoteka Campus <no-responder@colegio.edu.co>`, con el dominio verificado.

Sin dominio verificado, Resend solo entrega al correo dueño de la cuenta de Resend. Sirve para pruebas, no para el colegio.

Cloudinary (archivos)

1. Cree una cuenta en Cloudinary.
2. En el panel principal (**Dashboard** o **Settings** → **API Keys**) copie la variable de entorno de la API. Tiene la forma `cloudinary://API_KEY:API_SECRET@CLOUD_NAME` y es `CLOUDINARY_URL`.

Google AI Studio (IA)

1. Entre a Google AI Studio con una cuenta de Google.
2. En **Get API key**, cree una clave. Es `AI_API_KEY`.

La cuota gratuita se agota en días de mucho uso. Cuando pasa, el sistema sigue funcionando con las versiones sin IA.

4. Instalación local para desarrollo

Solo es necesaria para modificar el código o correr scripts de mantenimiento.

```
git clone https://github.com/Joan-Mora/ludoteka-campus.git
cd ludoteka-campus
npm install
cp apps/api/.env.example apps/api/.env
```

En Windows (PowerShell), el último comando es `Copy-Item apps/api/.env.example apps/api/.env`.

Abra `apps/api/.env` y complete al menos:

Variable	Valor
`DATABASE_URL`	La cadena del paso 2
`JWT_SECRET`	Una cadena aleatoria larga (ver abajo)
`SEED_PASSWORD`	Una clave fuerte para las cuentas de laboratorio

Y, si se van a usar: `AI_API_KEY`, `CLOUDINARY_URL`, `RESEND_API_KEY`, `MAIL_FROM` y `LAB_OTP_INBOX`.

Generar una cadena aleatoria para `JWT_SECRET` o `FACIAL_ENC_KEY`:

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

Arrancar:

```
npm run dev
```

- La web queda en `http://localhost:5173` y la API en `http://localhost:3000`.
- En desarrollo el código de ingreso aparece en la pantalla y en la consola de la API, así que no hace falta correo.

> **Atención:** si `DATABASE_URL` apunta a la base de producción, todo lo que haga en local (crear cursos, correr `seed`, `migrate` o `library:seed`) cambia producción. Para experimentar, cree un clúster aparte.

5. Instalación con Docker

La imagen del `Dockerfile` empaqueta la API y la web compiladas en un solo contenedor, igual que en producción: Node 22, solo dependencias de producción y un usuario sin privilegios. Escucha en el puerto 3000 y trae su propio healthcheck contra `/api/health`. No hace falta instalar Node; solo Docker Desktop (Windows o macOS) o Docker Engine con Compose v2.

Hay dos formas de levantarlo. En ambas, el contenedor lee `apps/api/.env` si existe, así que conviene crearlo antes (paso 4).

Opción A: contra la base de CockroachDB del `.env`.

```
docker compose up -d --build
```

- Abra `http://localhost:3000`. Para otro puerto: `APP_PORT=8080 docker compose up -d` (en PowerShell, `$env:APP_PORT=8080` antes del comando).
- `APP_PUBLIC_URL` y `CORS_ORIGIN` quedan en `http://localhost:3000` salvo que se definan en la terminal.
- Usa la base de `DATABASE_URL`: si es la de producción, todo lo que haga en el contenedor cambia producción.

Opción B: con una CockroachDB local, aislada y de demostración.

```
docker compose -f docker-compose.yml -f docker-compose.cockroach.yml up -d --build
```

- Levanta CockroachDB v26.2 de un solo nodo (modo inseguro, solo para pruebas), crea la base `ludoteka`, activa los índices vectoriales y arranca la aplicación contra ella. En el primer arranque se aplican las migraciones y se crea el colegio de demostración.
- La aplicación corre en modo desarrollo: el código de ingreso aparece en pantalla y no sale ningún correo.
- Las cuentas de laboratorio usan la clave `Ludoteka-Local-2026`, salvo que defina `SEED_PASSWORD` en la terminal.
- La consola de CockroachDB queda en `http://localhost:8080`.
- Las claves de IA y Cloudinary se siguen tomando del `.env`: los archivos que suba a la biblioteca van a la cuenta real de Cloudinary.

Comandos útiles (con la opción B, agregue `-f docker-compose.yml -f docker-compose.cockroach.yml` después de `docker compose`):

Comando	Qué hace
`docker compose ps`	Estado y salud de los contenedores
`docker compose logs -f app`	Log de la aplicación
`docker compose up -d --build`	Reconstruir después de cambiar el código
`docker compose down`	Detener (la base local se conserva)
`docker compose down -v`	Detener y borrar la base local para empezar de cero

Render no usa esta imagen: el despliegue sigue con el runtime de Node descrito en `render.yaml`.

6. Despliegue en Render

1. En Render, elija **New** → **Blueprint** y conecte el repositorio de GitHub.
2. Render lee `render.yaml` y propone el servicio `ludoteka-campus` (Node 22.14, build `npm ci --include=dev && npm run build`, inicio `npm start`, verificación `/api/health`).
3. Complete las variables marcadas para cargar a mano:

Variable	Valor
`DATABASE_URL`	Cadena de CockroachDB
`JWT_SECRET`	Cadena aleatoria larga
`APP_PUBLIC_URL`	URL pública, por ejemplo `https://ludoteka-campus.onrender.com`
`CORS_ORIGIN`	La misma URL pública
`AI_API_KEY`	Clave de Google AI Studio (o vacía)
`CLOUDINARY_URL`	Variable de Cloudinary
`RESEND_API_KEY`	Clave de Resend
`MAIL_FROM`	Remitente con el dominio verificado
`LAB_OTP_INBOX`	Un buzón real del equipo de sistemas (ver paso 6)
`SEED_PASSWORD`	Clave fuerte para las cuentas de laboratorio. **Obligatoria:** si queda vacía, se usa un valor por defecto que está escrito en el código público

4. Agregue a mano `FACIAL_ENC_KEY` con otra cadena aleatoria larga. No está en `render.yaml`; si falta, la llave del acceso facial se deriva de `JWT_SECRET` y cambiar ese secreto dejaría inservibles los rostros registrados.
5. Cree el servicio. El primer despliegue tarda unos minutos: instala, compila, aplica las migraciones y siembra el colegio de demostración.
6. Guarde todos los valores en el gestor de secretos del colegio. **Pierda `FACIAL_ENC_KEY` y se pierden los rostros registrados.**

Desde ese momento, cada push a la rama principal vuelve a desplegar. Si el despliegue nuevo falla, Render sigue sirviendo el anterior.

Plan gratuito: el servicio se duerme tras 15 minutos sin tráfico y la primera visita después tarda cerca de medio minuto. Para uso diario en el colegio conviene un plan pago, que no se duerme.

7. Primer ingreso y puesta en marcha

El primer arranque, con la base vacía, crea un colegio de demostración: el año 2026, los grados de sexto a undécimo, el grupo 6A, un curso de Matemáticas con 12 lecciones y siete cuentas de laboratorio con la clave de `SEED_PASSWORD`:

Correo	Rol
`admin@eduquest.local`	Administración
`rectoria@eduquest.local`	Rectoría
`coordinacion@eduquest.local`	Coordinación
`docente@eduquest.local`	Docente
`estudiante@eduquest.local`	Estudiante
`estudiante2@eduquest.local`	Estudiante (menor de edad)
`acudiente@eduquest.local`	Acudiente de ambos estudiantes

El dominio `@eduquest.local` no tiene buzón. Los códigos de ingreso de esas cuentas se envían a `LAB_OTP_INBOX`; sin esa variable, en producción no hay forma de leerlos.

Pasos:

1. Abra la URL pública y pulse **Ingresar**.
2. Entre con `admin@eduquest.local` y la clave de `SEED_PASSWORD`.
3. Escriba el código de 6 dígitos que llega a `LAB_OTP_INBOX`.
4. Acepte los documentos legales.
5. En **Usuarios**, cree la cuenta de administración real del colegio con un correo institucional. Le llegará una clave temporal.
6. Cierre sesión y entre con la cuenta real. Confirme que puede crear usuarios.
7. Configure la estructura del colegio en **Institución**: grados, grupos y materias (con la plantilla de estructura), carga docente, matrículas y cursos. El año lectivo y los periodos no tienen pantalla; se preparan con SQL (ver abajo). El paso a paso está en el Manual de usuario.
8. Cree o importe las cuentas de docentes, estudiantes y acudientes desde **Usuarios**.
9. Cuando ya no las necesite, **desactive** las cuentas de laboratorio desde **Usuarios**. No las borre en la base: el sistema recrea en cada arranque las que falten.

Periodos académicos: en una base nueva solo se crea el Periodo 1 del año de demostración, y la aplicación no tiene pantalla para crear periodos. Agréguelos en la consola SQL del clúster:

```
INSERT INTO periods (academic_year_id, name, position, starts_on, ends_on)
SELECT id, v.name, v.position, v.starts_on::DATE, v.ends_on::DATE
FROM academic_years
CROSS JOIN (VALUES
  ('Periodo 2', 2, '2026-05-01', '2026-07-15'),
```

```
  ('Periodo 3', 3, '2026-07-16', '2026-09-30'),
  ('Periodo 4', 4, '2026-10-01', '2026-11-30')
) AS v (name, position, starts_on, ends_on)
WHERE academic_years.name = '2026';
```

Ajuste el nombre del año y las fechas al calendario del colegio.

Biblioteca oficial (opcional): desde un equipo con la instalación local y las variables de producción, corra `npm run library:seed -w @eduquest/api`. Carga 60 guías propias por grado y materia y 9 documentos abiertos. Si Gemini agota la cuota, vuelva a correrlo otro día: retoma donde quedó.

8. Verificación

Prueba	Resultado esperado
Abrir `https://<su-dominio>/api/health`	`{"ok":true,"db":"up"}`
Abrir la URL pública	Página de inicio con el botón **Ingresar**
Ingresar con una cuenta real	Llega el código al correo y se abre el campus
Subir un archivo en la Biblioteca (cuenta docente)	El archivo queda publicado
Preguntar al Laboratorio IA (cuenta estudiante mayor de edad)	Responde con citas de la biblioteca
Registrar una passkey en Ajustes	El navegador pide Windows Hello, Face ID o huella

9. Dominio propio

1. En Render → servicio → **Settings** → **Custom Domains**, agregue el dominio (por ejemplo `campus.colegio.edu.co`).
2. Cree en el DNS el registro CNAME que Render indique y espere el certificado HTTPS.
3. Cambie `APP_PUBLIC_URL` y `CORS_ORIGIN` al dominio nuevo y guarde.
4. Las passkeys quedan atadas al dominio: las registradas con el dominio anterior dejan de servir y cada persona debe registrar la suya otra vez desde Ajustes.

10. Problemas frecuentes al instalar

Síntoma	Causa	Solución
El despliegue falla con un error de variables	Falta `DATABASE_URL` o `JWT_SECRET` (mínimo 8 caracteres)	Completar en Render → Environment
El despliegue falla en una migración	Versión de CockroachDB sin índices vectoriales o función deshabilitada	Ver el paso 2
No llega el código de `admin@eduquest.local`	`LAB_OTP_INBOX` vacía o Resend sin configurar	Definir ambas y volver a pedir el código
No llegan los códigos a cuentas reales	Dominio sin verificar en Resend	Verificar el dominio o revisar la tabla `email_outbox`
El Observatorio responde error de permisos	El usuario de `DATABASE_URL` no es administrador del clúster	Usar un usuario administrador
La biblioteca no acepta archivos	`CLOUDINARY_URL` vacía o mal copiada	Revisar la variable
Las passkeys fallan	`APP_PUBLIC_URL` no coincide con el dominio que ve el usuario	Ajustar `APP_PUBLIC_URL` y `CORS_ORIGIN`

Para la operación diaria (rotación de secretos, cambio de modelo de IA, monitoreo) siga Operación.